

Contents

Builder Design Pattern — From First Principles	1
1. The Problem (Before the Pattern)	1
2. Standard (GoF) Definition	2
3. How Builder Solves It (in words)	2
4. UML Diagram	3
5. Code Example	4
6. Types of Builder Pattern (FAANG Interview Depth)	5
7. Use Cases (Real World)	7
8. Builder vs Other Creational Patterns (Quick Interview Differentiator)	8
9. Memory Map (For Quick Recall Before an Interview)	8

Builder Design Pattern — From First Principles

1. The Problem (Before the Pattern)

Real-life analogy: Building a house

Imagine you hire a contractor and say: *“Build me a house.”*

The contractor asks: *“How many rooms? Garage? Swimming pool? Garden? Solar panels? What kind of roof?”*

If you had to describe your house using **one giant constructor call** with every possible option, it would look like this in code:

```
House house = new House(4, 2, true, false, true, "gable", 2, false, true, null, "brick", true);
```

Questions: - What does true in position 3 mean? Garage? Pool? - What if you don’t want a pool but want everything after it? - What if 2 years later we add “smart home wiring” as an option — do we now have a 13-parameter constructor?

This is called the **Telescoping Constructor Problem** — a classic anti-pattern where you keep adding overloaded constructors:

```
House(int rooms)
House(int rooms, int bathrooms)
House(int rooms, int bathrooms, boolean garage)
House(int rooms, int bathrooms, boolean garage, boolean pool)
// ... and so on, forever
```

Why this is genuinely bad engineering

Problem	Explanation
Unreadable calls	<code>new House(4, 2, true, false, true, "gable", 2, false, true, null, "brick", true)</code> tells you nothing about what each value means.
Error-prone	Swap two boolean parameters and the compiler won’t complain — you get a silently broken object.

Problem	Explanation
Doesn't scale	Every new optional feature multiplies the number of constructor overloads.
Forces you to pass "don't care" values	If you don't want a pool, you still must pass false explicitly, and remember its exact position.
No support for optional parameters with sane defaults	Java (unlike Python/Kotlin) has no native named/default parameters, so people fake it with overloading — badly.
Immutability becomes painful	If you want the object to be immutable (all fields final), you can't use setters after construction, so you're forced back into one huge constructor.

In one sentence: the problem Builder solves is — > *"How do I construct a complex object step-by-step, with many optional parts, in a readable and safe way, without a constructor explosion or a fragile pile of setters?"*

2. Standard (GoF) Definition

Builder is a creational design pattern that lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction process. — *Design Patterns: Elements of Reusable Object-Oriented Software* (Gang of Four, 1994)

Key phrase to remember: **"same construction process, different representations."** One assembly line (Director + steps), but depending on which Builder you plug in, you get a different final Product (e.g., a Toyota-builder vs a Ferrari-builder using the same "build engine, build wheels, build body" steps).

3. How Builder Solves It (in words)

Instead of one constructor doing everything at once, Builder:

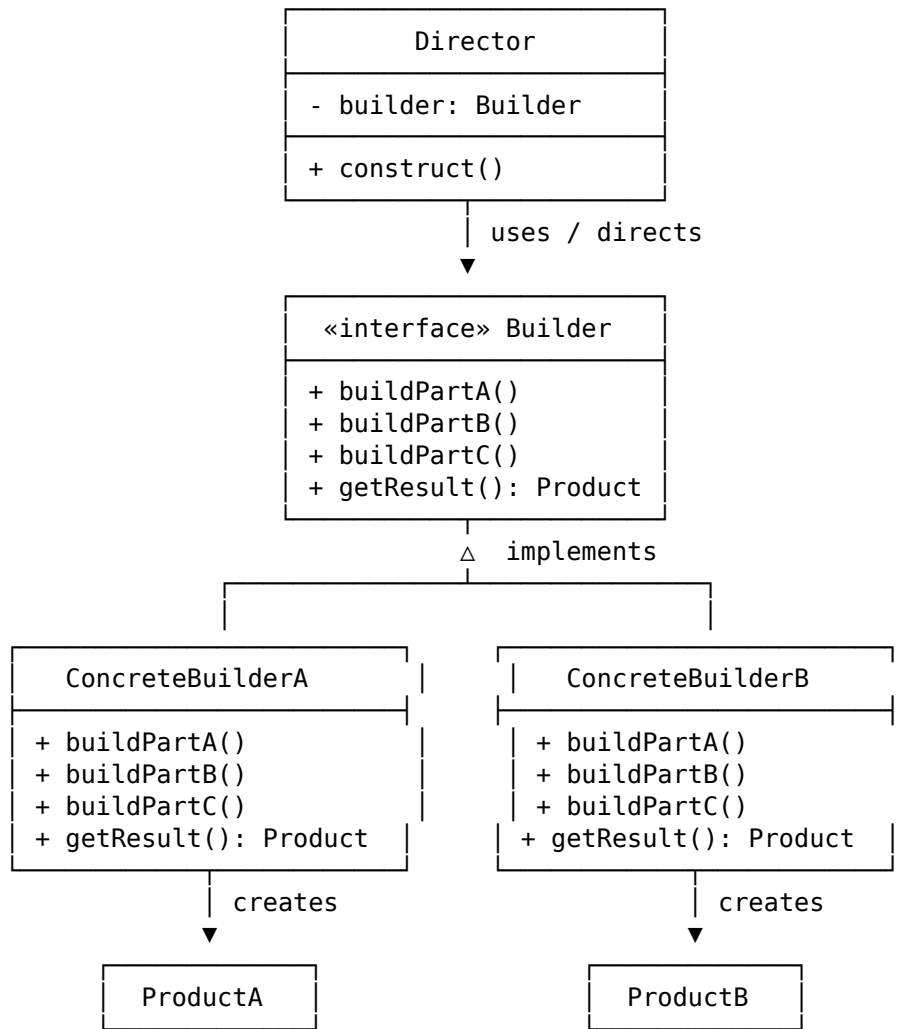
1. **Breaks construction into steps** — `setRooms()`, `setGarage()`, `setPool()`, etc.
2. **Lets you call only the steps you need** — skip what you don't want.
3. **Returns the finished object only when you explicitly ask for it** — via a `build()` method.
4. **Keeps the final object immutable** — the Builder mutates itself, not the final Product, and the Product's constructor (often `private/package-private`) is only called once, at the end.

Real-life analogy: Subway sandwich counter

At Subway, you don't order via one giant fixed command. You go step by step: 1. "White bread" → `chooseBread("white")` 2. "Chicken" → `addProtein("chicken")` 3. "Lettuce, tomato, no onions" → `addVeggies(...)` 4. "Toasted" → `toast(true)` 5. "That's it, wrap it up" → `build()`

The **sandwich artist (Builder)** knows how to assemble each part. You (the client) just say what you want, step by step, and only at the end do you get the finished sandwich (Product). You never see or touch a "half-built" sandwich object floating around in an inconsistent state.

4. UML Diagram



Relationships between the objects

Relationship	Type	Meaning
Director → Builder	Association (uses)	Director holds a reference to a Builder interface and calls its steps in a fixed sequence. Director does NOT know which concrete builder it has — polymorphism.
ConcreteBuilder → Builder	Realization (implements)	Each concrete builder implements the Builder interface's steps, but internally assembles its own specific Product differently.
ConcreteBuilder → Product	Dependency (creates)	Each concrete builder is responsible for instantiating and assembling one specific Product.

Relationship	Type	Meaning
Client → Director / Builder	Association	The client creates a ConcreteBuilder, optionally hands it to a Director, and retrieves the final Product via getResult().

The core insight: Director encapsulates the *order of steps* (the “recipe”), while Builder encapsulates the *how* of each step (the “ingredients and technique”). This separation is what lets the *same* recipe produce a wooden house or a stone house.

5. Code Example

5.1 Classic GoF style (Director + Builder interface)

```
// ----- Product -----
class House {
    private final int rooms;
    private final boolean garage;
    private final boolean pool;
    private final String roofType;

    // Package-private constructor - only Builder can call it
    House(int rooms, boolean garage, boolean pool, String roofType) {
        this.rooms = rooms;
        this.garage = garage;
        this.pool = pool;
        this.roofType = roofType;
    }

    @Override
    public String toString() {
        return "House{rooms=" + rooms + ", garage=" + garage +
            ", pool=" + pool + ", roof=" + roofType + "}";
    }
}

// ----- Builder Interface -----
interface HouseBuilder {
    HouseBuilder setRooms(int rooms);
    HouseBuilder setGarage(boolean garage);
    HouseBuilder setPool(boolean pool);
    HouseBuilder setRoofType(String type);
    House build();
}

// ----- Concrete Builder -----
class ConcreteHouseBuilder implements HouseBuilder {
    private int rooms;
    private boolean garage;
    private boolean pool;
    private String roofType;
}
```

```

    public HouseBuilder setRooms(int rooms) { this.rooms = rooms; return this; }
    public HouseBuilder setGarage(boolean garage) { this.garage = garage; return this; }
    public HouseBuilder setPool(boolean pool) { this.pool = pool; return this; }
    public HouseBuilder setRoofType(String type) { this.roofType = type; return this; }

    public House build() {
        return new House(rooms, garage, pool, roofType);
    }
}

// ----- Director (optional) -----
class HouseDirector {
    public House constructLuxuryHouse(HouseBuilder builder) {
        return builder.setRooms(6).setGarage(true).setPool(true).setRoofType("flat").build();
    }

    public House constructBasicHouse(HouseBuilder builder) {
        return builder.setRooms(2).setGarage(false).setPool(false).setRoofType("gable").build();
    }
}

// ----- Client -----
public class Main {
    public static void main(String[] args) {
        HouseDirector director = new HouseDirector();
        House luxury = director.constructLuxuryHouse(new ConcreteHouseBuilder());
        House basic = director.constructBasicHouse(new ConcreteHouseBuilder());
        System.out.println(luxury);
        System.out.println(basic);
    }
}

```

Notice: House has **no public constructor and no setters** — it's fully immutable once built. That's the payoff.

6. Types of Builder Pattern (FAANG Interview Depth)

Interviewers love asking “*what variations of Builder have you seen?*” — here are the four you must know.

Type 1 — Classic GoF Builder (Director-based)

Shown above. A Director encodes fixed “recipes” (constructLuxuryHouse, constructBasicHouse) reusing the same Builder steps.

Real-life analogy: A restaurant kitchen. The **Director** is the head chef who knows the recipe sequence (“sear the meat, then reduce the sauce, then plate”). The **Builder** is the line cook who actually knows *how* to sear, reduce, and plate for a specific dish. Swap the line cook (ConcreteBuilder) and you can make an Italian or Japanese version of the same recipe sequence.

When to use: When you have a few fixed, well-known “recipes” of construction you want to reuse across the app.

Type 2 — Fluent Builder (Method Chaining, No Director)

This is the **most common one you'll actually see in real codebases and asked in interviews** (e.g., “implement a fluent Builder for a Pizza class”).

```
class Pizza {
    private final String size;
    private final boolean cheese;
    private final boolean pepperoni;
    private final boolean mushrooms;

    private Pizza(Builder b) {
        this.size = b.size;
        this.cheese = b.cheese;
        this.pepperoni = b.pepperoni;
        this.mushrooms = b.mushrooms;
    }

    static class Builder {
        private final String size; // required
        private boolean cheese = false; // optional, default
        private boolean pepperoni = false; // optional, default
        private boolean mushrooms = false; // optional, default

        public Builder(String size) { this.size = size; } // required param forced here

        public Builder cheese(boolean val) { this.cheese = val; return this; }
        public Builder pepperoni(boolean val) { this.pepperoni = val; return this; }
        public Builder mushrooms(boolean val) { this.mushrooms = val; return this; }

        public Pizza build() { return new Pizza(this); }
    }
}

// Usage — reads almost like English:
Pizza pizza = new Pizza.Builder("Large")
    .cheese(true)
    .pepperoni(true)
    .build();
```

This is exactly the pattern **Joshua Bloch recommends in *Effective Java* (Item 2)** — required fields go into the Builder’s constructor, optional fields get chained setters, no Director needed.

Real-life analogy: Ordering a custom burger at In-N-Out “secret menu” style — you just keep saying “add this, add that” to the same order slip until you say “that’s my order,” and only then does the kitchen make it.

When to use: Almost always your default choice today. It’s what Lombok’s @Builder annotation and most modern Java/Kotlin/C# codebases generate.

Type 3 — Step Builder (Enforced Order via Interfaces)

A stricter version that uses a **chain of interfaces** so the compiler *forces* you to set fields in a specific order and prevents calling build() before mandatory fields are set.

```

interface SizeStep { CrustStep size(String size); }
interface CrustStep { ToppingStep crust(String crust); }
interface ToppingStep { BuildStep addTopping(String topping); BuildStep noToppings(); }
interface BuildStep { Pizza build(); }

class PizzaStepBuilder implements SizeStep, CrustStep, ToppingStep, BuildStep {
    private String size, crust;
    private List<String> toppings = new ArrayList<>();

    public static SizeStep create() { return new PizzaStepBuilder(); }

    public CrustStep size(String size) { this.size = size; return this; }
    public ToppingStep crust(String crust) { this.crust = crust; return this; }
    public BuildStep addTopping(String t) { toppings.add(t); return this; }
    public BuildStep noToppings() { return this; }
    public Pizza build() { return new Pizza(size, crust, toppings); }
}

// Usage - compiler enforces size -> crust -> toppings -> build order:
Pizza p = PizzaStepBuilder.create()
    .size("Large")
    .crust("Thin")
    .addTopping("Olives")
    .build();

```

Real-life analogy: Filling out a visa application form online where each page unlocks the next — you literally *cannot* reach “Submit” (build) without first completing “Personal Info” → “Passport Details” → “Travel Dates” in that exact order. The UI (compiler) physically prevents you from skipping a mandatory step.

When to use: When some fields are *mandatory* and you want compile-time (not runtime) safety that they were set, and there’s a natural sequence to the steps. Common in interview questions like “design a SQLQueryBuilder where WHERE must come after SELECT.”

Type 4 — Builder producing multiple representations from one process (true GoF intent)

This is the *original* motivating case from the GoF book: same director steps, wildly different products, e.g., an Exporter that can produce **HTML**, **PDF**, or **Plain Text** representations of the same Document object using the same sequence of buildTitle(), buildBody(), buildFooter() calls.

Real-life analogy: A single travel itinerary (the Director’s steps: “book flight, book hotel, book activities”) can be executed by a **Budget Travel Agent Builder** or a **Luxury Travel Agent Builder** — same steps, wildly different final trip (Product).

When to use: When you need the *same construction algorithm* to output *interchangeable but structurally different* representations.

7. Use Cases (Real World)

Use Case	Where it appears
StringBuilder / StringBuffer	Java's own standard library — step-by-step string assembly via <code>.append()</code> , <code>.build()</code> -like <code>.toString()</code> .
Lombok @Builder	Auto-generates a Fluent Builder for any POJO in Java projects.
Android AlertDialog.Builder	<code>new AlertDialog.Builder(context).setTitle(...).setMessage(...).set...</code>
OkHttpClient.Builder / Java's HttpClient.newBuilder()	Constructing complex HTTP clients with many optional configs (timeouts, interceptors, proxies).
SQL Query Builders (e.g., jOOQ, QueryDSL)	<code>select().from().where().orderBy().build()</code> — a Step Builder in disguise.
Protocol Buffers (protobuf) generated code	Every protobuf message class in Java gets a generated Builder inner class.
Test data builders in unit tests	Building complex test fixtures (<code>UserTestDataBuilder.aUser().withName("x").withRole("...</code> to avoid giant constructor calls in test code.
UI frameworks (SwiftUI-like DSLs, Jetpack Compose modifiers)	Chained <code>.padding().background().clickable()</code> style — conceptually a fluent builder over a UI node tree.

8. Builder vs Other Creational Patterns (Quick Interview Differentiator)

Pattern	Difference from Builder
Factory Method	Creates an object in one call , decides <i>which subclass</i> to instantiate. Builder assembles a single complex object over multiple calls .
Abstract Factory	Creates families of related objects (e.g., a whole UI kit: Button + Checkbox + Scrollbar for "Dark Theme"). Builder creates one object's internal parts step by step.
Prototype	Creates a new object by cloning an existing one. Builder creates a new object from scratch, part by part .

9. Memory Map (For Quick Recall Before an Interview)

BUILDER PATTERN – MEMORY MAP

PROBLEM	Telescoping constructors / too many optional params / need immutability
DEFINITION (say this verbatim)	"Construct complex objects step-by-step; same process, different representations"
KEY PLAYERS	(4 roles – memorize as "DBCP")

- └─ Director → knows the ORDER of steps (optional role)
 - └─ Builder (iface) → defines the STEPS (contract)
 - └─ ConcreteBuilder → implements steps, HOLDS partial state
 - └─ Product → final immutable object, private/no-arg-less constructor
 - ─ RELATIONSHIPS
 - └─ Director --uses--> Builder (interface)
 - └─ ConcreteBuilder --implements--> Builder
 - └─ ConcreteBuilder --creates--> Product
 - ─ 4 TYPES (mnemonic: "Classic Fluent Steps Multi")
 - └─ 1. Classic (GoF) → Director-driven, fixed recipes [restaurant chef analogy]
 - └─ 2. Fluent → chained setters, no Director [Subway/burger analogy] □ most common
 - └─ 3. Step Builder → interfaces force mandatory order [visa form analogy]
 - └─ 4. Multi-Rep → same steps → different output types [budget vs luxury travel agent]
 - ─ REAL WORLD
 - └─ StringBuilder, Lombok @Builder, OkHttp/HttpClient.Builder, protobuf Builders, AlertDialog.Builder
 - ─ DIFFERENTIATE FROM
 - └─ Factory Method → one call, picks subclass
 - └─ Abstract Factory → families of related objects
 - └─ Prototype → clone existing object
-

One-line summary to say out loud in an interview:

"Builder separates the construction of a complex object from its representation, so the same step-by-step process can produce different results — solving telescoping constructors while enabling immutability and readable, fluent object creation."